# huffman coding

mahmood

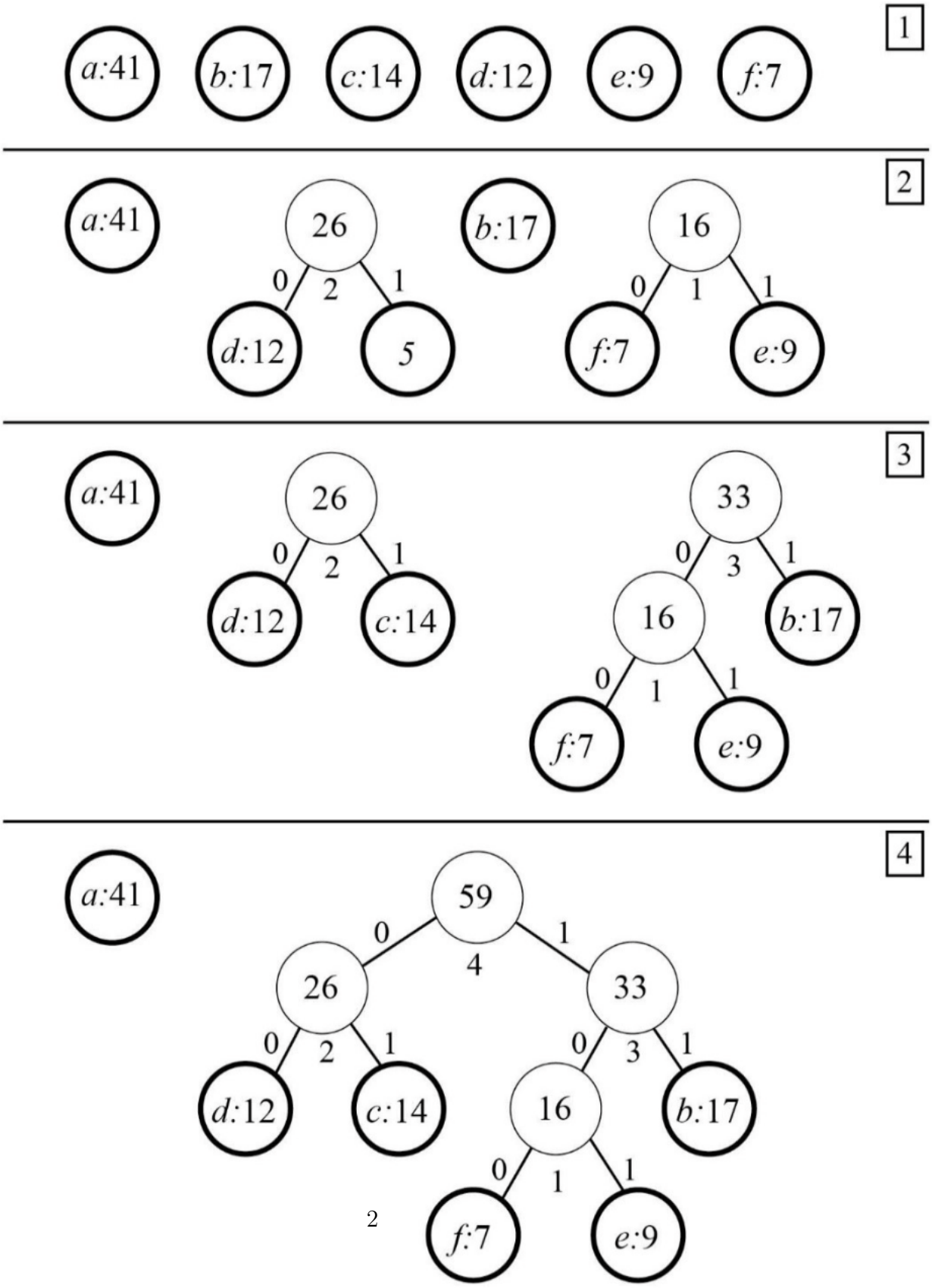
April 2, 2023

**Huffman code** is a particular type of optimal prefix code that is commonly used for lossless data compression

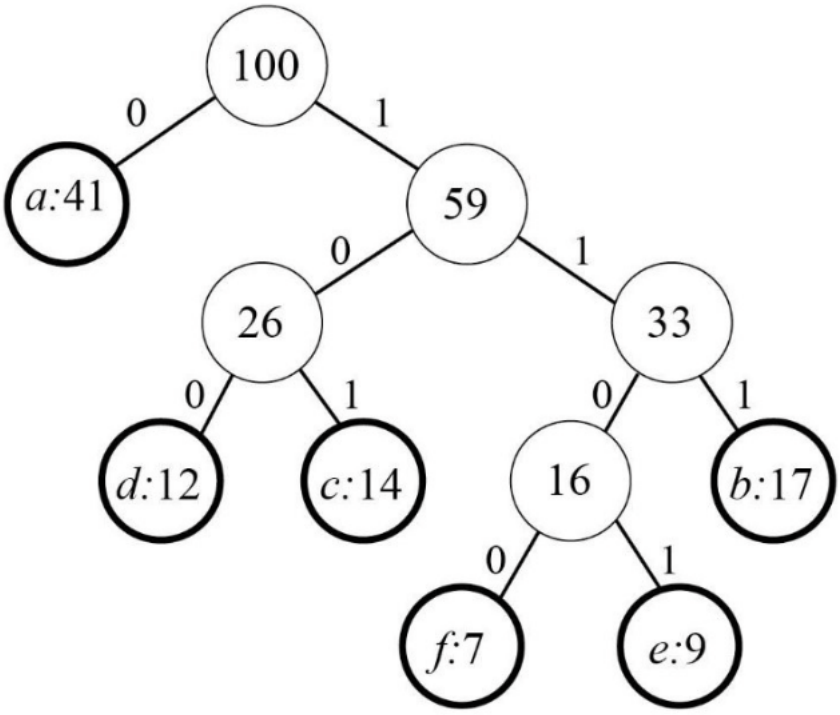
**Example 0.0.1.** the problem that huffman code solves is the following:
input: a leaf array that corresponds to a file $F$
output: optimal code tree of $F$, $T_{out}$
consider the following leaf array

| char | freq |
|---|---|
| a | 41 |
| b | 17 |
| c | 14 |
| d | 12 |
| e | 9 |
| f | 7 |

this is the application of **huffman coding** to the leaf array


1
a:41
b:17
c:14
d:12
e:9
f:7
2
a:41
26
0
2
1
d:12
5
b:17
16
0
1
1
f:7
e:9
3
a:41
26
0
2
1
d:12
c:14
33
0
3
1
16
b:17
0
1
1
f:7
e:9
4
a:41
59
0
4
1
26
33
0
2
1
0
3
1
d:12
c:14
16
b:17
0
1
1
f:7
e:9

so we end up with the following code tree:




```
Huffman(D):
    PQ ← InitializePQ(D)
    for i ← 1 to n − 2 do
        v_x ← PQ.extractMin()
        v_y ← PQ.extractMin()
        v_z ← new vertex(f_x + f_y)
        v_z.left ← v_x
        v_z.right ← v_y
        PQ.insert(p_z)
    v_x ← PQ.extractMin()
    v_y ← PQ.extractMin()
    r ← new vertex(f_x + f_y)
    r.left ← v_x
    r.right ← v_y
    T_out ← r
    return T_out
```

the process begins with the leaf nodes containing the frequencies of the symbol they represent. then, the process takes the two nodes with smallest frequency, and creates a new internal node having these two nodes as children. the frequency of the new node is set to the sum of the frequencies of

the children. we then apply the process again, on the new internal node and on the remaining nodes (i.e., we exclude the two leaf nodes), we repeat this process until only one node remains, which is the root of the huffman tree. the algorithm we proposed uses a priority queue where the node with lowest frequency is given highest priority:

1. create a leaf node for each symbol and add it to the priority queue.

2. while there is more than one node in the queue:

   (a) remove the two nodes of highest priority (lowest frequency) from the queue

   (b) create a new internal node with these two nodes as children and with frequency equal to the sum of the two nodes' frequencies.

   (c) add the new node to the queue.

3. the remaining node is the root node and the tree is complete.